JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.concurrent

Class CompletableFuture<T>

java.lang.Object
  java.util.concurrent.CompletableFuture<T>

All Implemented Interfaces:

CompletionStage<T>, Future<T>

```
public class CompletableFuture<T>
extends Object
implements Future<T>, CompletionStage<T>
```

A Future that may be explicitly completed (setting its value and status), and may be used as a CompletionStage, supporting dependent functions and actions that trigger upon its completion.

When two or more threads attempt to complete, completeExceptionally, or cancel a CompletableFuture, only one of them succeeds.

In addition to these and related methods for directly manipulating status and results, CompletableFuture implements interface CompletionStage with the following policies:

- Actions supplied for dependent completions of *non-async* methods may be performed by the thread that completes the current CompletableFuture, or by any other caller of a completion method.
- All *async* methods without an explicit Executor argument are performed using the ForkJoinPool.commonPool() (unless it does not support a parallelism level of at least two, in which case, a new Thread is created to run each task). To simplify monitoring, debugging, and tracking, all generated asynchronous tasks are instances of the marker interface CompletableFuture.AsynchronousCompletionTask.
- All CompletionStage methods are implemented independently of other public methods, so the behavior of one method is not impacted by overrides of others in subclasses.

CompletableFuture also implements `Future` with the following policies:

- Since (unlike `FutureTask`) this class has no direct control over the computation that causes it to be completed, cancellation is treated as just another form of exceptional completion. Method `cancel` has the same effect as `completeExceptionally(new CancellationException())`. Method `isCompletedExceptionally()` can be used to determine if a CompletableFuture completed in any exceptional fashion.
- In case of exceptional completion with a CompletionException, methods `get()` and `get(long, TimeUnit)` throw an `ExecutionException` with the same cause as held in the corresponding CompletionException. To simplify usage in most contexts, this class also defines methods `join()` and `getNow(T)` that instead throw the CompletionException directly in these cases.

Since:

1.8

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description
static interface	**CompletableFuture.AsynchronousCompletionTask**	A marker interface identifying asynchronous tasks produced by async methods.

Constructor Summary

Constructors

Constructor	Description
CompletableFuture()	Creates a new incomplete CompletableFuture.

Method Summary

Concrete Methods

Modifier and Type	Method	Descriptio

CompletableFuture<Void>	acceptEither(CompletionStage<? extends T> other, Consumer<? super T> action)	Returns a Completio either this stage com executed correspon argument action.
CompletableFuture<Void>	acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action)	Returns a Completio either this stage com executed default as execution correspon argument action.
CompletableFuture<Void>	acceptEitherAsync(CompletionStage<? extends T> other, Consumer<? super T> action, Executor executor)	Returns a Completio either this stage com executed executor, correspon argument function.
static CompletableFuture<Void>	allOf(CompletableFuture<?>... cfs)	Returns a Completa completed given Con complete.
static CompletableFuture<Object>	anyOf(CompletableFuture<?>... cfs)	Returns a Completa completed given Con complete, result.
<U> CompletableFuture<U>	applyToEither(CompletionStage<? extends T> other, Function<? super T,U> fn)	Returns a Completio either this stage com executed correspon

		argument function.
<U> **CompletableFuture**<U>	**applyToEitherAsync**(**CompletionStage**<? extends **T**> other, **Function**<? super **T**,U> fn)	Returns a Completio either this stage com executed default as execution correspon argument function.
<U> **CompletableFuture**<U>	**applyToEitherAsync**(**CompletionStage**<? extends **T**> other, **Function**<? super **T**,U> fn, **Executor** executor)	Returns a Completio either this stage com executed executor, correspon argument function.
boolean	**cancel**(boolean mayInterruptIfRunning)	If not alre completes Completa **Cancella**
boolean	**complete**(**T** value)	If not alre sets the v **get()** and to the giv
static <U> **CompletableFuture**<U>	**completedFuture**(U value)	Returns a Completa already c given valu
boolean	**completeExceptionally**(**Throwable** ex)	If not alre causes inv and relate throw the
CompletableFuture<**T**>	**exceptionally**(**Function**<**Throwable**,? extends **T**> fn)	Returns a Completa completed Completa completes the given

		exception Completa completio completes otherwise Completa completes returned also comp the same
T	get()	Waits if ne future to returns its
T	get(long timeout, TimeUnit unit)	Waits if ne the given to comple returns its available.
T	getNow(T valueIfAbsent)	Returns th throws an exception returns th valueIfAbs
int	getNumberOfDependents()	Returns th number o Completa completio completio Completa
<U> CompletableFuture<U>	handle(BiFunction<? super T,Throwable,? extends U> fn)	Returns a Completio this stage normally executed result and argument function.
<U> CompletableFuture<U>	handleAsync(BiFunction<? super T,Throwable,? extends U> fn)	Returns a Completio this stage normally executed default as execution

		stage's re as argume function.
<U> **CompletableFuture**<U>	**handleAsync**(**BiFunction**<? super **T**,**Throwable**,? extends U> fn, **Executor** executor)	Returns a Completio this stage normally executed executor, result and argument function.
boolean	**isCancelled**()	Returns t Completa cancelled completed
boolean	**isCompletedExceptionally**()	Returns t Completa completed any way.
boolean	**isDone**()	Returns t any fashio exception cancellati
T	**join**()	Returns th when com (unchecke completed
void	**obtrudeException**(**Throwable** ex)	Forcibly c invocation and relate throw the whether c completed
void	**obtrudeValue**(**T** value)	Forcibly s value subs by metho methods, already co
CompletableFuture<**Void**>	**runAfterBoth**(**CompletionStage**<? > other, **Runnable** action)	Returns a Completio this and th

		stage both normally, action.
`CompletableFuture<Void>`	`runAfterBothAsync(CompletionStage<?> other, Runnable action)`	Returns a Completio this and th stage com executes t using this asynchron facility.
`CompletableFuture<Void>`	`runAfterBothAsync(CompletionStage<?> other, Runnable action, Executor executor)`	Returns a Completio this and th stage com executes t using the
`CompletableFuture<Void>`	`runAfterEither(CompletionStage<?> other, Runnable action)`	Returns a Completio either this stage com executes t
`CompletableFuture<Void>`	`runAfterEitherAsync(CompletionStage<?> other, Runnable action)`	Returns a Completio either this stage com executes t using this asynchron facility.
`CompletableFuture<Void>`	`runAfterEitherAsync(CompletionStage<?> other, Runnable action, Executor executor)`	Returns a Completio either this stage com executes t using the
`static CompletableFuture<Void>`	`runAsync(Runnable runnable)`	Returns a Completa asynchron a task run ForkJoin after it ru
`static CompletableFuture<Void>`		

	runAsync(**Runnable** runnable, **Executor** executor)	Returns a Completa asynchron a task run executor a given acti
static <U> **CompletableFuture**<U>	**supplyAsync**(**Supplier**<U> supplier)	Returns a Completa asynchron a task run **ForkJoin** with the v calling the
static <U> **CompletableFuture**<U>	**supplyAsync**(**Supplier**<U> supplier, **Executor** executor)	Returns a Completa asynchron a task run executor obtained Supplier.
CompletableFuture<**Void**>	**thenAccept**(**Consumer**<? super **T**> action)	Returns a Completio this stage normally, this stage argument action.
CompletableFuture<**Void**>	**thenAcceptAsync**(**Consumer**<? super **T**> action)	Returns a Completio this stage normally, this stage asynchron facility, wi result as t supplied a
CompletableFuture<**Void**>	**thenAcceptAsync**(**Consumer**<? super **T**> action, **Executor** executor)	Returns a Completio this stage normally, the suppli this stage argument action.

<U> **CompletableFuture**<**Void**>	**thenAcceptBoth**(**CompletionStage**<? extends U> other, **BiConsumer**<? super **T**,? super U> action)	Returns a Completio this and t stage both normally, the two re to the sup
<U> **CompletableFuture**<**Void**>	**thenAcceptBothAsync**(**CompletionStage**<? extends U> other, **BiConsumer**<? super **T**,? super U> action)	Returns a Completio this and t stage com executed default as execution two result the suppli
<U> **CompletableFuture**<**Void**>	**thenAcceptBothAsync**(**CompletionStage**<? extends U> other, **BiConsumer**<? super **T**,? super U> action, **Executor** executor)	Returns a Completio this and t stage com executed executor, results as supplied f
<U> **CompletableFuture**<U>	**thenApply**(**Function**<? super **T**,? extends U> fn)	Returns a Completio this stage normally, this stage argument function.
<U> **CompletableFuture**<U>	**thenApplyAsync**(**Function**<? super **T**,? extends U> fn)	Returns a Completio this stage normally, this stage asynchron facility, wi result as t supplied f
<U> **CompletableFuture**<U>	**thenApplyAsync**(**Function**<? super **T**,? extends U> fn, **Executor** executor)	Returns a Completio this stage normally,

		the suppli this stage argument function.
<U,V> **CompletableFuture**<V>	**thenCombine**(**CompletionStage**<? extends U> other, **BiFunction**<? super **T**,? super U,? extends V> fn)	Returns a Completio this and th stage both normally, the two re to the sup
<U,V> **CompletableFuture**<V>	**thenCombineAsync**(**CompletionStage**<? extends U> other, **BiFunction**<? super **T**,? super U,? extends V> fn)	Returns a Completio this and th stage com executed default as execution two result the suppli
<U,V> **CompletableFuture**<V>	**thenCombineAsync**(**CompletionStage**<? extends U> other, **BiFunction**<? super **T**,? super U,? extends V> fn, **Executor** executor)	Returns a Completio this and th stage com executed executor, results as supplied f
<U> **CompletableFuture**<U>	**thenCompose**(**Function**<? super **T**,? extends **CompletionStage**<U>> fn)	Returns a Completio this stage normally, this stage the suppli
<U> **CompletableFuture**<U>	**thenComposeAsync**(**Function**<? super **T**,? extends **CompletionStage**<U>> fn)	Returns a Completio this stage normally, this stage asynchron facility, wi argument function.
<U> **CompletableFuture**<U>		

	thenComposeAsync(**Function**<? super **T**,? extends **CompletionStage**<U>> fn, **Executor** executor)	Returns a Completio this stage normally, the suppli this stage argument function.
CompletableFuture<**Void**>	**thenRun**(**Runnable** action)	Returns a Completio this stage normally, action.
CompletableFuture<**Void**>	**thenRunAsync**(**Runnable** action)	Returns a Completio this stage normally, action usi default as execution
CompletableFuture<**Void**>	**thenRunAsync**(**Runnable** action, **Executor** executor)	Returns a Completio this stage normally, action usi Executor.
CompletableFuture<**T**>	**toCompletableFuture**()	Returns th Completa
String	**toString**()	Returns a this Comp well as its
CompletableFuture<**T**>	**whenComplete**(**BiConsumer**<? super **T**,? super **Throwable**> action)	Returns a Completio same resu this stage given acti completes
CompletableFuture<**T**>	**whenCompleteAsync**(**BiConsumer**<? super **T**,? super **Throwable**> action)	Returns a Completio same resu this stage given acti stage's de

		execution stage com
`CompletableFuture<T>`	`whenCompleteAsync(BiConsumer<? super T,? super Throwable> action, Executor executor)`	Returns a Completio same resu this stage given acti supplied E stage com

Methods inherited from class java.lang.Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, wait, wait, wait

Constructor Detail

CompletableFuture

```
public CompletableFuture()
```

Creates a new incomplete CompletableFuture.

Method Detail

supplyAsync

```
public static <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the `ForkJoinPool.commonPool()` with the value obtained by calling the given Supplier.

Type Parameters:

`U - the function's return type`

Parameters:

`supplier - a function returning the value to be used to complete the returned CompletableFuture`

Returns:

`the new CompletableFuture`

supplyAsync

```
public static <U> CompletableFuture<U> supplyAsync(Supplier<U> supplier,
                                                   Executor executor)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the given executor with the value obtained by calling the given Supplier.

Type Parameters:

`U - the function's return type`

Parameters:

`supplier - a function returning the value to be used to complete the returned CompletableFuture`

`executor - the executor to use for asynchronous execution`

Returns:

`the new CompletableFuture`

runAsync

```
public static CompletableFuture<Void> runAsync(Runnable runnable)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the `ForkJoinPool.commonPool()` after it runs the given action.

Parameters:

`runnable - the action to run before completing the returned CompletableFuture`

Returns:

`the new CompletableFuture`

runAsync

```
public static CompletableFuture<Void> runAsync(Runnable runnable,
                                               Executor executor)
```

Returns a new CompletableFuture that is asynchronously completed by a task running in the given executor after it runs the given action.

Parameters:

`runnable - the action to run before completing the returned CompletableFuture`

`executor - the executor to use for asynchronous execution`

Returns:

the new CompletableFuture

completedFuture

```
public static <U> CompletableFuture<U> completedFuture(U value)
```

Returns a new CompletableFuture that is already completed with the given value.

Type Parameters:

U - the type of the value

Parameters:

value - the value

Returns:

the completed CompletableFuture

isDone

```
public boolean isDone()
```

Returns `true` if completed in any fashion: normally, exceptionally, or via cancellation.

Specified by:

isDone in interface Future<T>

Returns:

true if completed

get

```
public T get()
      throws InterruptedException,
             ExecutionException
```

Waits if necessary for this future to complete, and then returns its result.

Specified by:

get in interface Future<T>

Returns:

the result value

Throws:

CancellationException - if this future was cancelled

ExecutionException - if this future completed exceptionally

InterruptedException - if the current thread was interrupted while waiting

get

```
public T get(long timeout,
             TimeUnit unit)
      throws InterruptedException,
             ExecutionException,
             TimeoutException
```

Waits if necessary for at most the given time for this future to complete, and then returns its result, if available.

Specified by:

get in interface Future<T>

Parameters:

timeout - the maximum time to wait

unit - the time unit of the timeout argument

Returns:

the result value

Throws:

CancellationException - if this future was cancelled

ExecutionException - if this future completed exceptionally

InterruptedException - if the current thread was interrupted while waiting

TimeoutException - if the wait timed out

join

```
public T join()
```

Returns the result value when complete, or throws an (unchecked) exception if completed exceptionally. To better conform with the use of common functional forms, if a computation involved in the completion of this CompletableFuture threw an exception, this method throws an (unchecked) CompletionException with the underlying exception as its cause.

Returns:

the result value

Throws:

CancellationException - if the computation was cancelled

CompletionException - if this future completed exceptionally or a completion computation threw an exception

getNow

```
public T getNow(T valueIfAbsent)
```

Returns the result value (or throws any encountered exception) if completed, else returns the given valueIfAbsent.

Parameters:

valueIfAbsent - the value to return if not completed

Returns:

the result value, if completed, else the given valueIfAbsent

Throws:

CancellationException - if the computation was cancelled

CompletionException - if this future completed exceptionally or a completion computation threw an exception

complete

```
public boolean complete(T value)
```

If not already completed, sets the value returned by `get()` and related methods to the given value.

Parameters:

value - the result value

Returns:

true if this invocation caused this CompletableFuture to transition to a completed state, else false

completeExceptionally

```
public boolean completeExceptionally(Throwable ex)
```

If not already completed, causes invocations of `get()` and related methods to throw the given exception.

Parameters:

ex - the exception

Returns:

true if this invocation caused this CompletableFuture to transition to a completed state, else false

thenApply

public <U> CompletableFuture<U> thenApply(Function<? super T,? extends U> fn)

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenApply in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenApplyAsync

public <U> CompletableFuture<U> thenApplyAsync(Function<? super T,? extends U> fn

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenApplyAsync in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenApplyAsync

```
public <U> CompletableFuture<U> thenApplyAsync(Function<? super T,? extends U> fn
                                               Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenApplyAsync in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

fn - the function to use to compute the value of the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenAccept

```
public CompletableFuture<Void> thenAccept(Consumer<? super T> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAccept in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptAsync

```
public CompletableFuture<Void> thenAcceptAsync(Consumer<? super T> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAcceptAsync in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptAsync

```
public CompletableFuture<Void> thenAcceptAsync(Consumer<? super T> action,
                                               Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAcceptAsync in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenRun

```
public CompletableFuture<Void> thenRun(Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, executes the given action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenRun in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenRunAsync

```
public CompletableFuture<Void> thenRunAsync(Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, executes the given action using this stage's default asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenRunAsync in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenRunAsync

```
public CompletableFuture<Void> thenRunAsync(Runnable action,
                                            Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, executes the given action using the supplied Executor. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenRunAsync in interface CompletionStage<T>

Parameters:

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenCombine

```
public <U,V> CompletableFuture<V> thenCombine(CompletionStage<? extends U> other,
                                              BiFunction<? super T,? super U,? ex
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenCombine in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

V - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenCombineAsync

```
public <U,V> CompletableFuture<V> thenCombineAsync(CompletionStage<? extends U> o
                                                   BiFunction<? super T,? super U
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenCombineAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

V - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

thenCombineAsync

```
public <U,V> CompletableFuture<V> thenCombineAsync(CompletionStage<? extends U> o
                                                   BiFunction<? super T,? super U
                                                   Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenCombineAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

V - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenAcceptBoth

```
public <U> CompletableFuture<Void> thenAcceptBoth(CompletionStage<? extends U> ot
                                                  BiConsumer<? super T,? super U>
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage both complete normally, is executed with the two results as arguments to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenAcceptBoth in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptBothAsync

```
public <U> CompletableFuture<Void> thenAcceptBothAsync(CompletionStage<? extends
                                                       BiConsumer<? super T,? sup
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the two results as arguments to the supplied action.

Specified by:

thenAcceptBothAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the other CompletionStage's result

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

thenAcceptBothAsync

```
public <U> CompletableFuture<Void> thenAcceptBothAsync(CompletionStage<? extends
                                                       BiConsumer<? super T,? sup
                                                       Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage complete normally, is executed using the supplied executor, with the two results as arguments to the supplied function.

Specified by:

`thenAcceptBothAsync` in interface `CompletionStage<T>`

Type Parameters:

`U - the type of the other CompletionStage's result`

Parameters:

`other - the other CompletionStage`

`action - the action to perform before completing the returned CompletionStage`

`executor - the executor to use for asynchronous execution`

Returns:

`the new CompletionStage`

runAfterBoth

```
public CompletableFuture<Void> runAfterBoth(CompletionStage<?> other,
                                            Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage both complete normally, executes the given action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

`runAfterBoth` in interface `CompletionStage<T>`

Parameters:

`other - the other CompletionStage`

`action - the action to perform before completing the returned CompletionStage`

Returns:

`the new CompletionStage`

runAfterBothAsync

```
public CompletableFuture<Void> runAfterBothAsync(CompletionStage<?> other,
                                                 Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage complete normally, executes the given action using this stage's default asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterBothAsync in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterBothAsync

```
public CompletableFuture<Void> runAfterBothAsync(CompletionStage<?> other,
                                                 Runnable action,
                                                 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this and the other given stage complete normally, executes the given action using the supplied executor. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterBothAsync in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

applyToEither

```
public <U> CompletableFuture<U> applyToEither(CompletionStage<? extends T> other,
                                              Function<? super T,U> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

applyToEither in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

applyToEitherAsync

```
public <U> CompletableFuture<U> applyToEitherAsync(CompletionStage<? extends T> o
                                                   Function<? super T,U> fn)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

applyToEitherAsync in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

applyToEitherAsync

```
public <U> CompletableFuture<U> applyToEitherAsync(CompletionStage<? extends T> c
                                                   Function<? super T,U> fn,
                                                   Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

applyToEitherAsync in interface CompletionStage<T>

Type Parameters:

U - the function's return type

Parameters:

other - the other CompletionStage

fn - the function to use to compute the value of the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

acceptEither

```
public CompletableFuture<Void> acceptEither(CompletionStage<? extends T> other,
                                            Consumer<? super T> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed with the corresponding result as argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

acceptEither in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

acceptEitherAsync

```
public CompletableFuture<Void> acceptEitherAsync(CompletionStage<? extends T> oth
                                                 Consumer<? super T> action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using this stage's default asynchronous execution facility, with the corresponding result as argument to the supplied action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

acceptEitherAsync in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

acceptEitherAsync

```
public CompletableFuture<Void> acceptEitherAsync(CompletionStage<? extends T> oth
                                                 Consumer<? super T> action,
                                                 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, is executed using the supplied executor, with the corresponding result as argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

acceptEitherAsync in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

runAfterEither

```
public CompletableFuture<Void> runAfterEither(CompletionStage<?> other,
                                              Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterEither in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterEitherAsync

```
public CompletableFuture<Void> runAfterEitherAsync(CompletionStage<?> other,
                                                   Runnable action)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using this stage's default asynchronous execution facility. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterEitherAsync in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

Returns:

the new CompletionStage

runAfterEitherAsync

```
public CompletableFuture<Void> runAfterEitherAsync(CompletionStage<?> other,
                                                   Runnable action,
                                                   Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when either this or the other given stage complete normally, executes the given action using the supplied executor. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

runAfterEitherAsync in interface CompletionStage<T>

Parameters:

other - the other CompletionStage

action - the action to perform before completing the returned CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the new CompletionStage

thenCompose

```
public <U> CompletableFuture<U> thenCompose(Function<? super T,? extends Completi
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed with this stage as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenCompose in interface CompletionStage<T>

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function returning a new CompletionStage

Returns:

the CompletionStage

thenComposeAsync

```
public <U> CompletableFuture<U> thenComposeAsync(Function<? super T,? extends Com
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using this stage's default asynchronous execution facility, with this stage as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenComposeAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function returning a new CompletionStage

Returns:

the CompletionStage

thenComposeAsync

```
public <U> CompletableFuture<U> thenComposeAsync(Function<? super T,? extends Com
                                                 Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes normally, is executed using the supplied Executor, with this stage's result as the argument to the supplied function. See the CompletionStage documentation for rules covering exceptional completion.

Specified by:

thenComposeAsync in interface CompletionStage<T>

Type Parameters:

U - the type of the returned CompletionStage's result

Parameters:

fn - the function returning a new CompletionStage

executor - the executor to use for asynchronous execution

Returns:

the CompletionStage

whenComplete

public CompletableFuture<T> whenComplete(BiConsumer<? super T,? super Throwable>

Description copied from interface: CompletionStage

Returns a new CompletionStage with the same result or exception as this stage, that executes the given action when this stage completes.

When this stage is complete, the given action is invoked with the result (or null if none) and the exception (or null if none) of this stage as arguments. The returned stage is completed when the action returns. If the supplied action itself encounters an exception, then the returned stage exceptionally completes with this exception unless this stage also completed exceptionally.

Specified by:

whenComplete in interface CompletionStage<T>

Parameters:

action - the action to perform

Returns:

the new CompletionStage

whenCompleteAsync

public CompletableFuture<T> whenCompleteAsync(BiConsumer<? super T,? super Throwa

Description copied from interface: CompletionStage

Returns a new CompletionStage with the same result or exception as this stage, that executes the given action using this stage's default asynchronous execution facility when this stage completes.

When this stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments. The returned stage is completed when the action returns. If the supplied action itself encounters an exception, then the returned stage exceptionally completes with this exception unless this stage also completed exceptionally.

Specified by:

`whenCompleteAsync` in interface `CompletionStage<T>`

Parameters:

`action` - the action to perform

Returns:

the new CompletionStage

whenCompleteAsync

```
public CompletableFuture<T> whenCompleteAsync(BiConsumer<? super T,? super Throwa
                                              Executor executor)
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage with the same result or exception as this stage, that executes the given action using the supplied Executor when this stage completes.

When this stage is complete, the given action is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments. The returned stage is completed when the action returns. If the supplied action itself encounters an exception, then the returned stage exceptionally completes with this exception unless this stage also completed exceptionally.

Specified by:

`whenCompleteAsync` in interface `CompletionStage<T>`

Parameters:

`action` - the action to perform

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

handle

```
public <U> CompletableFuture<U> handle(BiFunction<? super T,Throwable,? extends U
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Specified by:

`handle` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

handleAsync

```
public <U> CompletableFuture<U> handleAsync(BiFunction<? super T,Throwable,? exte
```

Description copied from interface: `CompletionStage`

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using this stage's default asynchronous execution facility, with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Specified by:

`handleAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

Returns:

the new CompletionStage

handleAsync

```
public <U> CompletableFuture<U> handleAsync(BiFunction<? super T,Throwable,? exte
                                            Executor executor)
```

Description copied from interface: CompletionStage

Returns a new CompletionStage that, when this stage completes either normally or exceptionally, is executed using the supplied executor, with this stage's result and exception as arguments to the supplied function.

When this stage is complete, the given function is invoked with the result (or `null` if none) and the exception (or `null` if none) of this stage as arguments, and the function's result is used to complete the returned stage.

Specified by:

`handleAsync` in interface `CompletionStage<T>`

Type Parameters:

`U` - the function's return type

Parameters:

`fn` - the function to use to compute the value of the returned CompletionStage

`executor` - the executor to use for asynchronous execution

Returns:

the new CompletionStage

toCompletableFuture

```
public CompletableFuture<T> toCompletableFuture()
```

Returns this CompletableFuture.

Specified by:

`toCompletableFuture` in interface `CompletionStage<T>`

Returns:

this CompletableFuture

exceptionally

```
public CompletableFuture<T> exceptionally(Function<Throwable,? extends T> fn)
```

Returns a new CompletableFuture that is completed when this CompletableFuture completes, with the result of the given function of the exception triggering this CompletableFuture's completion when it completes exceptionally; otherwise, if this CompletableFuture completes normally, then

the returned CompletableFuture also completes normally with the same value. Note: More flexible versions of this functionality are available using methods `whenComplete` and `handle`.

Specified by:

`exceptionally` in interface `CompletionStage<T>`

Parameters:

`fn - the function to use to compute the value of the returned CompletableFuture if this CompletableFuture completed exceptionally`

Returns:

`the new CompletableFuture`

allOf

```
public static CompletableFuture<Void> allOf(CompletableFuture<?>... cfs)
```

Returns a new CompletableFuture that is completed when all of the given CompletableFutures complete. If any of the given CompletableFutures complete exceptionally, then the returned CompletableFuture also does so, with a CompletionException holding this exception as its cause. Otherwise, the results, if any, of the given CompletableFutures are not reflected in the returned CompletableFuture, but may be obtained by inspecting them individually. If no CompletableFutures are provided, returns a CompletableFuture completed with the value `null`.

Among the applications of this method is to await completion of a set of independent CompletableFutures before continuing a program, as in: `CompletableFuture.allOf(c1, c2, c3).join();`.

Parameters:

`cfs - the CompletableFutures`

Returns:

`a new CompletableFuture that is completed when all of the given CompletableFutures complete`

Throws:

`NullPointerException - if the array or any of its elements are null`

anyOf

```
public static CompletableFuture<Object> anyOf(CompletableFuture<?>... cfs)
```

Returns a new CompletableFuture that is completed when any of the given CompletableFutures complete, with the same result. Otherwise, if it

completed exceptionally, the returned CompletableFuture also does so, with a CompletionException holding this exception as its cause. If no CompletableFutures are provided, returns an incomplete CompletableFuture.

Parameters:

`cfs - the CompletableFutures`

Returns:

`a new CompletableFuture that is completed with the result or exception of any of the given CompletableFutures when one completes`

Throws:

`NullPointerException - if the array or any of its elements are null`

cancel

```
public boolean cancel(boolean mayInterruptIfRunning)
```

If not already completed, completes this CompletableFuture with a `CancellationException`. Dependent CompletableFutures that have not already completed will also complete exceptionally, with a `CompletionException` caused by this `CancellationException`.

Specified by:

`cancel in interface Future<T>`

Parameters:

`mayInterruptIfRunning - this value has no effect in this implementation because interrupts are not used to control processing.`

Returns:

`true if this task is now cancelled`

isCancelled

```
public boolean isCancelled()
```

Returns `true` if this CompletableFuture was cancelled before it completed normally.

Specified by:

`isCancelled` in interface `Future<T>`

Returns:

`true if this CompletableFuture was cancelled before it completed normally`

isCompletedExceptionally

```
public boolean isCompletedExceptionally()
```

Returns `true` if this CompletableFuture completed exceptionally, in any way. Possible causes include cancellation, explicit invocation of `completeExceptionally`, and abrupt termination of a CompletionStage action.

Returns:

`true if this CompletableFuture completed exceptionally`

obtrudeValue

```
public void obtrudeValue(T value)
```

Forcibly sets or resets the value subsequently returned by method `get()` and related methods, whether or not already completed. This method is designed for use only in error recovery actions, and even in such situations may result in ongoing dependent completions using established versus overwritten outcomes.

Parameters:

`value - the completion value`

obtrudeException

```
public void obtrudeException(Throwable ex)
```

Forcibly causes subsequent invocations of method `get()` and related methods to throw the given exception, whether or not already completed. This method is designed for use only in error recovery actions, and even in

such situations may result in ongoing dependent completions using established versus overwritten outcomes.

Parameters:

`ex - the exception`

Throws:

`NullPointerException - if the exception is null`

getNumberOfDependents

```
public int getNumberOfDependents()
```

Returns the estimated number of CompletableFutures whose completions are awaiting completion of this CompletableFuture. This method is designed for use in monitoring system state, not for synchronization control.

Returns:

`the number of dependent CompletableFutures`

toString

```
public String toString()
```

Returns a string identifying this CompletableFuture, as well as its completion state. The state, in brackets, contains the String `"Completed Normally"` or the String `"Completed Exceptionally"`, or the String `"Not completed"` followed by the number of CompletableFutures dependent upon its completion, if any.

Overrides:

`toString in class Object`

Returns:

```
a string identifying this CompletableFuture, as well as its state
```


Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.